SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Design & Optimisation Task : 10%

Total Marks: 50

Annexure A

Code of Conduct:

I P.Gowtham Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinaryaction.

Signature of the student: P.Gowtham Reddy

Annexure A

Debugging & Optimisation Task

1. Secure Password Storage in Web Applications

A web application stores user passwords and needs to protect them against

database breaches.

A. Explain why storing plain text passwords is insecure.

B. Suggest a secure method for storing passwords and justify your choice.

ANSWER:

Scenario

A Python program using the **SHA-256** hashing algorithm generates different hash values for the same input due to implementation errors.

A. Errors Identified

1. Incorrect **message padding** (missing padding bits or message length).
2. Wrong **block processing** (not processing 512-bit blocks correctly).
3. Incorrect **bitwise operations** (rotate and shift operations implemented incorrectly).
4. **Endianness mismatch** (using little-endian instead of big-endian format).

B. Fixes

- Apply **SHA-256 padding** correctly by appending `1`, followed by `0`s, and the message length.
- Process data in **512-bit blocks**.
- Use correct **bitwise rotation (ROTR)** and XOR operations.
- Read/write data in **big-endian** format as specified by the SHA standard.

C. Optimization for Large Files (≥50 MB)

- Read files in **small chunks (e.g., 4 KB or 8 KB)** instead of loading the entire file into memory.
- Use Python's built-in `hashlib.sha256()`, which is optimized and faster.
- Avoid unnecessary memory allocation during hashing.

D. Security Comparison: SHA vs Checksum

SHA Hash Function	Basic Checksum
Cryptographically secure	Not secure
Detects intentional modifications	Detects only simple errors
Collision-resistant	Easy to forge
Used for digital signatures and security	Used mainly for error detection

---

Conclusion

By correcting **padding**, **block processing**, **bitwise operations**, and **endianness**, the SHA implementation produces consistent and secure hash values. Using **chunk-based processing** and Python's hashlib improves performance for large files. Compared to basic checksums, **SHA provides much stronger data integrity and security**, making it suitable for modern applications.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.
Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

ANSWER:

Scenario

A Python-based **SSL/TLS handshake simulation** experiences delays and occasional handshake failures due to errors in certificate validation and key exchange.

A. Debugging Issues

- Verify that the **server certificate** is valid, trusted, and not expired.
- Ensure the **public/private keys** used in key exchange are correct.
- Check that both client and server support the **same TLS version** and **cipher suite**.
- Handle handshake exceptions and invalid certificates properly.

B. Handshake Optimization

- Use **TLS 1.3**, which reduces the number of handshake messages.
- Enable **faster key exchange** using **Elliptic Curve Diffie-Hellman (ECDHE)**.
- Minimize unnecessary certificate verification operations.
- Reduce network round trips to lower latency.

C. Session Resumption

- Reuse previously established **session keys** using **Session IDs** or **Session Tickets**.
- Skip the full handshake for returning clients.
- Reduce CPU usage and connection setup time.

D. Performance vs Security Trade-offs

Optimization	Performance Benefit	Security Impact
TLS 1.3	Faster handshake	Strong security
Session Resumption	Lower latency	Secure if session keys are protected
ECDHE Key Exchange	Faster key exchange	Provides Forward Secrecy
Certificate Validation	Slight delay	Essential for authentication

---

Conclusion

By fixing **certificate validation** and **key exchange** issues, using **TLS 1.3**, enabling **session resumption**, and adopting **ECDHE**, the SSL/TLS simulation becomes faster and more reliable while maintaining strong security for secure communication.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.
Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

ANSWER:

Scenario

A Python implementation of the **Digital Signature Algorithm (DSA)** generates invalid signatures during verification due to errors in parameter selection and random number generation.

A. Identify Issues

- Incorrect **DSA parameters** (p, q, g) are used.
- Weak or repeated **random number (k)** generation.
- Errors in **modular arithmetic** during signature creation.
- Public/private keys are not generated correctly.

B. Fix the Errors

- Use valid DSA parameters (p, q, g) as per standards.
- Generate a **unique, cryptographically secure random value (k)** for every signature.
- Implement correct **signature generation** and **verification** formulas.
- Verify the public key before checking the signature.

C. Optimize Execution

- Use **fast modular exponentiation** (pow(base, exp, mod) in Python).
- Reuse validated public parameters.
- Use optimized cryptographic libraries instead of manual calculations.
- Reduce unnecessary computations during verification.

D. Impact of Randomness on Security

Good Randomness	Poor Randomness
Unique random k for each signature	Reused or predictable k
Strong security	Private key may be exposed
Prevents forgery	Vulnerable to attacks
Reliable digital signatures	Invalid or insecure signatures

Conclusion

Using **correct DSA parameters**, **secure random number generation**, and **optimized modular arithmetic** ensures valid and efficient digital signatures. High-quality randomness is essential because weak or repeated random values can compromise the **private key** and the overall security of the DSA system.

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.
Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

ANSWER:

Scenario

A Python-based **Key Management System (KMS)** fails to securely store and retrieve cryptographic keys, making the system vulnerable to attacks.

A. Identify Flaws

- Cryptographic keys are stored in **plaintext**.
- Weak or no encryption is used for key storage.
- Poor access control allows unauthorized users to access keys.
- Keys are not backed up or protected properly.

B. Fix Key Lifecycle Issues

- Generate keys using a **cryptographically secure random number generator**.
- Distribute keys through **secure encrypted channels**.
- Rotate (change) keys periodically.
- Revoke and securely destroy expired or compromised keys.

C. Optimize Secure Storage

- Encrypt keys using a **Master Key** or **AES encryption**.
- Store keys in a secure **Key Management System (KMS)** or **Hardware Security Module (HSM)**.
- Implement **Role-Based Access Control (RBAC)** and multi-factor authentication (MFA).
- Maintain audit logs for key access and usage.

D. Impact of Poor Key Management

Good Key Management	Poor Key Management
Keys stored securely	Keys stored in plaintext
Regular key rotation	Old keys reused
Strong access control	Unauthorized access
Secure cryptographic system	Entire system can be compromised

Conclusion

By **encrypting stored keys**, implementing **secure key generation, distribution, rotation**, and enforcing **strict access control**, the Key Management System becomes secure and reliable. Poor key management can expose encryption keys, allowing attackers to decrypt sensitive data and compromise the entire cryptographic system.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.
Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).

Compare the optimized system with insecure file transfer methods and justify the improvements.
ANSWER:

Scenario

A Python-based **SFTP-like system** experiences slow file transfers and occasional data corruption while transferring large files (≥100 MB).

A. Debug Issues

- Ensure **encryption and decryption** use the same algorithm, key, and IV.
- Verify sender and receiver are synchronized during data transfer.
- Check for packet loss and transmission errors.

B. Fix Packet Handling and Integrity

- Transfer files in **fixed-size packets**.
- Add a **SHA-256 hash** or **Message Authentication Code (MAC)** to each packet.
- Verify packet integrity before accepting the data.
- Retransmit corrupted or missing packets.

C. Optimize Large File Transfer

- Transfer files in **chunks (e.g., 4 KB or 8 KB)** instead of loading the entire file into memory.
- Use efficient encryption such as **AES**.
- Enable buffering and parallel processing where possible.
- Compress files before transfer to reduce transmission time.

D. Comparison: Optimized SFTP vs Insecure File Transfer

Optimized SFTP	Insecure File Transfer (FTP)
Data encrypted	Data sent in plaintext
SHA-256/MAC verifies integrity	No integrity protection
Secure authentication	Weak authentication
Protects against data theft	Vulnerable to attacks

Conclusion

By synchronizing **encryption/decryption**, verifying packet integrity with **SHA-256 or MAC**, and using **chunk-based transfer with AES encryption**, the SFTP-like system

becomes faster, more reliable, and secure. Compared to insecure file transfer methods like FTP, the optimized system provides **confidentiality, integrity, and secure authentication**, making it suitable for transferring sensitive files.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation